

AWS Jam 实战题目总结

网络、权限与安全专题

本文档汇总了四道 AWS Jam 实战题目的
问题背景、根本原因、操作步骤与知识点，
适用于 AWS 认证备考与实战参考。

涉及服务	Amazon VPC, S3, IAM, KMS, Lambda
难度	中级
类别	网络 / 权限 / 安全

2026 年 3 月 9 日

目录

1 题目一：VPC Lambda 无法访问互联网

1.1 背景

开发了一个 AWS Lambda 函数，需要同时访问：

- 互联网资源（<http://example.com>）
- VPC 内私有子网上 EC2 实例的 Web 服务

函数配置为 VPC Lambda，部署后执行成功但 CloudWatch Logs 出现网络错误。

1.2 根本原因

⚠ 根本原因

Lambda 被放置在公有子网（Public Subnet）中。

虽然公有子网的路由表指向 Internet Gateway（IGW），但 **Lambda ENI 永远不会被分配公网 IP**，IGW 找不到对应的公网 IP 映射，导致出站流量被丢弃。

1.2.1 核心原理：IGW 的工作机制

Internet Gateway 进行的是 **一对一 NAT**，要求资源拥有公网 IP：

资源类型	公有子网中能获得公网 IP	能否访问互联网
EC2 实例	✓（自动分配或 EIP）	✓
NAT Gateway	✓（必须绑定 EIP）	✓
Lambda ENI	×（AWS 不分配）	×

1.2.2 私有子网为何可行

私有子网路由表指向 NAT Gateway，NAT Gateway 自身持有 EIP，代替 Lambda 访问互联网：

```
1 # 私有子网路由表（正确）
2 Destination      Target
3 10.0.0.0/16      local
4 0.0.0.0/0        nat-xxxxxxx    <- NAT Gateway, 有公网 IP
5
6 # 公有子网路由表（对 Lambda 无效）
7 Destination      Target
8 10.0.0.0/16      local
```

```
9 0.0.0.0/0      igw-xxxxxxxx    <- IGW, Lambda ENI 无公网 IP
```

Listing 1: 私有子网路由表示例

1.3 操作步骤

✓ 解决方案

1. 打开 **CloudWatch Logs**, 确认 `/aws/lambda/MyFunction` 中存在连接超时等网络错误。
2. 打开 **Lambda Console** → 选择 `MyFunction` → **Configuration** → **VPC**。
3. 点击 **Edit**, 将子网从公有子网更换为私有子网。
 - 私有子网特征: 路由表中 `0.0.0.0/0` → `nat-*`
 - 公有子网特征: 路由表中 `0.0.0.0/0` → `igw-*`
4. 点击 **Save**, 重新执行函数并验证日志无报错。

💡 核心知识点

VPC Lambda 网络访问规则:

- Lambda 放在私有子网 + 私有子网有 NAT Gateway 路由 → 可访问互联网 & 私有资源
- Lambda 放在公有子网 → 只能访问私有资源, 无法访问互联网
- 安全组需放行出站 80/443 端口 (互联网) 及 EC2 对应端口

2 题目二：Lambda 无法读写 S3 对象

2.1 背景

通过 CloudFormation 创建了 S3 Bucket 和 IAM Role，用于同账号的 Lambda 函数进行 CRUD 操作。应用团队反馈 Lambda 无法对 Bucket 进行任何读写操作。

2.2 根本原因



根本原因

IAM Policy 中，对象级操作的 Resource ARN 缺少 /* 后缀。

S3 ARN 在 IAM 中具有两种语义，对象级操作必须使用 `arn:aws:s3:::bucket-name/*` 才能生效。

2.2.1 S3 ARN 两种语义对照

ARN 格式	代表含义	对应操作
<code>arn:aws:s3:::my-bucket</code>	桶本身	<code>s3:ListBucket</code> 等桶级操作
<code>arn:aws:s3:::my-bucket/*</code>	桶内所有对象	<code>s3:GetObject</code> 、 <code>s3:PutObject</code> 等

2.2.2 错误的 IAM Policy

```
1 {  
2   "Effect": "Allow",  
3   "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],  
4   "Resource": "arn:aws:s3:::my-bucket"  
5 }
```

Listing 2: 错误配置（缺少 /*）

2.2.3 正确的 IAM Policy

```
1 {  
2   "Effect": "Allow",  
3   "Action": ["s3:ListBucket"],  
4   "Resource": "arn:aws:s3:::my-bucket"  
5 },  
6 {  
7   "Effect": "Allow",
```

```
8   "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
9   "Resource": "arn:aws:s3:::my-bucket/*"
10 }
```

Listing 3: 正确配置

2.3 操作步骤

✓ 解决方案

1. 打开 **IAM Console** → **Roles** → 找到目标 Role。
2. 检查 **Trust relationships**, 确认 `lambda.amazonaws.com` 为信任主体。
3. 展开 **Permissions** 中的自定义 Policy, 找到对象级操作语句。
4. 将 **Resource** 的值从 `arn:aws:s3:::bucket-name` 修改为 `arn:aws:s3:::bucket-name/*`。
5. 保存后重新测试 Lambda 函数。

2.3.1 CloudFormation 正确写法

```
1 - Effect: Allow
2   Action:
3     - s3:ListBucket
4   Resource:
5     - !GetAtt MyBucket.Arn          # arn:aws:s3:::my-bucket
6
7 - Effect: Allow
8   Action:
9     - s3:GetObject
10    - s3:PutObject
11    - s3:DeleteObject
12   Resource:
13     - !Sub "${MyBucket.Arn}/*"      # arn:aws:s3:::my-bucket/*
```

Listing 4: CloudFormation IAM Policy 正确示例

 核心知识点

口诀：桶操作用桶 ARN，对象操作必须加 /*

IAM Policy 还需注意 Trust Policy 中服务主体正确性：

- Lambda 使用的 Role: "Service": "lambda.amazonaws.com"
- EC2 使用的 Role: "Service": "ec2.amazonaws.com"

3 题目三：KMS Key 管理员被锁出

3.1 背景

作为 AWS 安全管理员，在开发阶段创建了一个 KMS Key。开发阶段结束后，发现无法再管理该 KMS Key，且公司规定 KMS 权限仅通过 Key Policy 管理，不依赖 IAM Policy。

3.2 根本原因

⚠ 根本原因

KMS Key Policy 被修改后，当前 IAM Role 未被包含在 Key Administrators 列表中，导致失去管理权限。

KMS 的权限模型与其他 AWS 服务不同：Key Policy 是**第一道门**，IAM Policy 必须先被 Key Policy 允许才能生效。

3.2.1 KMS 权限模型对比

服务	权限控制方式	说明
S3、SQS 等	Identity-based Policy 与 Resource-based Policy 共同评估	通常取并集，显式 Deny 优先
KMS	Key Policy 是前提条件，IAM Policy 在其基础上补充	Key Policy 优先

3.2.2 被锁出的 Key Policy 示例

```
1 {
2   "Sid": "KeyAdmins",
3   "Effect": "Allow",
4   "Principal": {
5     "AWS": "arn:aws:iam::111122223333:role/dev-team-role"
6   },
7   "Action": ["kms:*"],
8   "Resource": "*"
9 }
```

Listing 5: 错误的 Key Policy（管理员角色不含当前用户）

3.3 操作步骤

✓ 解决方案

1. 在 Console 右上角切换到临时管理员角色 jam-aws-tmp-admin-role (Switch Role)。
2. 打开 **KMS Console** → 找到 jam-data-encryption-key。
3. 点击 **Key policy** 标签页 → 点击 **Edit**。
4. 在 `KeyAdministrators` 语句的 `Principal.AWS` 数组中, 添加 `JamTask3SecurityAdminRoleArn`:

```
1 {  
2   "Sid": "KeyAdministrators",  
3   "Effect": "Allow",  
4   "Principal": {  
5     "AWS": [  
6       "arn:aws:iam::111122223333:role/jam-aws-tmp-admin-role",  
7       "arn:aws:iam::111122223333:role/JamTask3SecurityAdminRole"  
8     ]  
9   },  
10  "Action": [  
11    "kms:Create*", "kms:Describe*", "kms:Enable*",  
12    "kms:List*",   "kms:Put*",       "kms:Update*",  
13    "kms:Revoke*", "kms:Disable*",  "kms:Get*",  
14    "kms:Delete*", "kms:ScheduleKeyDeletion", "kms:CancelKeyDeletion"  
15  ],  
16  "Resource": "*"   
17 }
```

Listing 6: 修复后的 Key Policy

💡 核心知识点

KMS Key Policy 最佳实践:

- 永远在 Key Policy 中保留至少一个可靠的管理员角色。
- 若既未保留账户级入口, 也没有其他可管理主体, KMS Key 可能变得 **unmanageable**; 此时可能需要账户紧急管理员介入, 严重时需联系 AWS Support。

- 生产环境建议保留至少两个管理员身份，防止单点锁出。
- `kms:*` 权限不等于 IAM 的管理员权限，必须显式写入 Key Policy。

4 题目四：S3 跨账号访问控制过于宽松

4.1 背景

提供了一个 S3 Bucket 供外部 AWS 账号用户访问数据。要求仅有被分配了特定角色 S3CrossAccountAccess 的用户才能访问，但实际上该外部账号下的所有用户都能访问。

4.2 根本原因

⚠ 根本原因

Bucket Policy 中 Principal 使用了外部账号的 root，这并不表示“只允许对方 Root 用户”，而是把访问权限委派给整个外部账号。随后该账号管理员可以再通过 IAM Policy 将权限授予多个用户或角色，因此超出了“只允许特定角色访问”的要求。

4.2.1 Principal 写法的语义差异

Principal 写法	谁能访问	是否
ACCOUNT_ID:root	委派给整个外部账号，由对方账号再授予具体主体	×
ACCOUNT_ID:role/S3CrossAccountAccess	仅该特定角色	✓
ACCOUNT_ID:user/specific-user	仅该特定用户	✓

4.2.2 错误的 Bucket Policy

```

1 {
2   "Effect": "Allow",
3   "Principal": {
4     "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:root"
5   },
6   "Action": ["s3:GetObject", "s3:ListBucket"],
7   "Resource": [
8     "arn:aws:s3:::your-bucket",
9     "arn:aws:s3:::your-bucket/*"
10  ]
11 }
```

Listing 7: 错误配置（Principal 为 root）

4.2.3 修复后的 Bucket Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "AllowSpecificRoleOnly",
6       "Effect": "Allow",
7       "Principal": {
8         "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:role/
9           S3CrossAccountAccess"
10      },
11      "Action": ["s3:GetObject", "s3:ListBucket"],
12      "Resource": [
13        "arn:aws:s3:::your-bucket",
14        "arn:aws:s3:::your-bucket/*"
15      ]
16    }
17  ]
18 }
```

Listing 8: 正确配置（Principal 精确到角色）

4.3 操作步骤

✓ 解决方案

1. 打开 **S3 Console** → 找到目标 Bucket。
2. 点击 **Permissions** 标签页 → **Bucket policy** → 点击 **Edit**。
3. 将 Principal 从：
 - "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:root"修改为：
 - "AWS": "arn:aws:iam::EXTERNAL_ACCOUNT_ID:role/S3CrossAccountAccess"
4. 点击 **Save changes**。
5. 验证：外部账号中未获得该角色权限、或未假设该角色的主体应收到 **Access Denied**；仅 **S3CrossAccountAccess** 角色可正常访问。

 核心知识点**最小权限原则 (Principle of Least Privilege):**

- 跨账号访问时, **Principal** 应精确到具体的 **Role/User ARN**。
- 避免使用 **:root** 进行跨账号授权; 只有在你明确要把权限委派给整个外部账号时才这样做。
- 对于敏感数据, 还应结合 **Condition** 加入 MFA、IP、VPC Endpoint 等限制条件。

5 综合知识点汇总

5.1 AWS 权限控制体系

服务	权限控制层	注意事项
Lambda + VPC	子网路由 (NAT/IGW)	Lambda ENI 无公网 IP
S3 + IAM	IAM Policy Resource ARN	对象操作需 /* 后缀
KMS	Key Policy 优先	须显式包含管理员 ARN
S3 跨账号	Bucket Policy Principal	避免使用 :root 委派整个账号

5.2 常见错误速查表

症状	常见原因	修复方向
Lambda 连接超时	在公有子网, 无 NAT	改为私有子网
S3 GetObject 403	IAM Resource 缺 /*	补全 ARN 后缀
KMS 无权限	Role 不在 Key Policy 中	编辑 Key Policy
跨账号访问过宽	Principal 为 root	改为精确 Role ARN

5.3 最佳实践总结

1. **VPC Lambda:** 始终将 Lambda 放置在私有子网, 通过 NAT Gateway 访问互联网。
2. **IAM Policy:** S3 对象级操作的 Resource 必须使用 `arn:aws:s3:::bucket/*` 格式。
3. **KMS Key Policy:** 始终保留至少两个管理员 ARN, 防止自我锁出。
4. **跨账号权限:** Principal 精确到角色或用户, 遵循最小权限原则。
5. **IAM 信任策略:** Lambda 角色的 Trust Policy 中服务主体必须为 `lambda.amazonaws.com`。